

Group Report on the CS7CS3 Advanced Software Engineering Group Project

Group: 8

Mansi Jaiswal - mjaiswal@tcd.ie,
Reetik Kirankumar Mathankar - mathankr@tcd.ie,
Marie Courtine - courtinm@tcd.ie,
Marta Fraioli - fraiolim@tcd.ie,
Vinol Tauro - taurov@tcd.ie,
Niraj Bharambe - bharambn@tcd.ie,
Tanmay Dilip Birari - birarit@tcd.ie,
Sriram Kirthivas - kirthivs@tcd.ie,
Pravigya Jain - jainpr@tcd.ie,
Ajinkya Taraneekar - taraneeka@tcd.ie

This report is intended for you to provide a reflection on the use of eXtreme Programming in your group project.

Instructions:

1. Please give a “real” assessment of the actual benefits and drawbacks of using XP for your project under each of the 12 core practices of XP. Indicate when each category was not applicable, and why¹.
2. Change the <group number> in the heading above to your group number
3. Change the <list of group members> to be your group members
4. Change the name of this file to replace the ‘N’ in the current ‘GroupN’ with your group number
5. Submit on Blackboard – note ONE SUBMISSION is sufficient for your whole group.

1. Whole Team

“All the contributors to an XP project – developers, business analysts, testers, etc. – work together in an open space, members of one team....”

How was this applied in your team?

The team worked primarily in person at the Phoenix House laboratory provided by Trinity College Dublin, an open-plan space that naturally supported XP's *whole team* principle. The layout allowed pairs to work independently while remaining physically

¹ It is highly unlikely (I would suggest not possible) that any Generative AI tool will be able to tell you what your experiences were in your group. I say this to indicate that a GenAI response to these questions will be treated with considerable skepticism on my part.

close enough to collaborate across pairs when needed. We also booked the small meeting rooms in the lab at least once a week for short stand-up style meetings, using the screen and whiteboard to review progress. Early on, during the first reading week, we experimented with remote meetings on Google Meet, but participation was lower and the sessions felt slower, so we canceled these additional meetings.

Benefits as experienced in your team

Working in the same physical space made collaboration natural and low-friction: if a pair hit a blocker, help was immediately available without scheduling overhead. Over time, the lab became our shared workspace even for other courses. Spending so much time together also created a strong sense of mutual support that went beyond the purely technical: the team naturally started meeting socially outside university as well, which meaningfully boosted morale and resilience throughout the project. This felt particularly noteworthy given that, as a cohort shaped by COVID, most of us had grown accustomed to working from home, yet the shared space made coming in feel natural rather than an obligation.

Drawbacks as experienced in your team

The open-plan environment was occasionally disruptive, as the lab was shared with other teams and background noise could make it harder to focus. The early attempt during the first reading week at remote collaboration via Google Meet revealed that our team worked significantly less effectively online, which limited flexibility during periods when meeting in person was difficult.

Lessons learned

Co-location proved far more effective than remote collaboration for our team, and its benefits extended well beyond productivity. In future projects, we would prioritise securing a dedicated physical workspace from the outset. The experience also challenged assumptions many of us held about remote work being the default, the social and psychological value of shared physical presence turned out to be one of the most important factors in keeping the team motivated and cohesive.

2. Planning Game

“Planning is continuous and progressive. Every two weeks, for the next two weeks, developers estimate the cost of candidate features, and customers select those features to be implemented based upon cost and business value”

How was this applied in your team?

Planning decisions were made collectively during weekly team meetings, where tasks were discussed and estimated in working days, a unit the team agreed upon as roughly consistent across members. For pair tracking, we used a google sheet. In this sheet it is possible to see the primary person of the task in one column and the other member of the pair that discovers the task.

Week 1	
Sriram	Pravigya
Mansi	Tanmay
Marie	Marta
Niraj	Ajinkya
Reetik	Vinol

For task tracking, the team went through an evolution of tools. We used Jira during all the projects. During the first section of the project, we duplicated this project management and task allocation by using google sheet but this created some confusion so we decided to stick to Jira. Each Monday, we took care of moving the task in the Kanban Board. So each Monday it was possible to follow the pace of each pair. In the Jira, the task is assigned to the primary person in the pair.

☐ SCRUM-46 EPIC: Developer Environment Setup	TERMINÉ(E) ▼	23 janv.	-	=	NS
☒ SCRUM-47 EPIC: Cost Cutting - GCP	TERMINÉ(E) ▼	23 janv.	-	=	M
☒ SCRUM-48 EPIC: Keycloak Migration to GKE	TERMINÉ(E) ▼	23 janv.	-	=	AT
☒ SCRUM-49 EPIC: Dashboard Migration to GKE	TERMINÉ(E) ▼	23 janv.	-	=	NS
☒ SCRUM-50 EPIC: GKE VM Scheduling	TERMINÉ(E) ▼	23 janv.	-	=	T
☒ SCRUM-51 EPIC: ClickHouse on GKE	TERMINÉ(E) ▼	23 janv.	-	=	NS
☒ SCRUM-52 EPIC: Services Boilerplate & Deploy	TERMINÉ(E) ▼	23 janv.	-	=	M
☒ SCRUM-53 EPIC: Bug Fixes	TERMINÉ(E) ▼	23 janv.	-	=	PI
☒ SCRUM-54 EPIC: Data ETL Service	TERMINÉ(E) ▼	23 janv.	-	=	T
☒ SCRUM-55 EPIC: Observability Service	TERMINÉ(E) ▼	23 janv.	-	=	NS
☒ SCRUM-56 EPIC: Airflow on GKE	TERMINÉ(E) ▼	28 janv.	-	=	TB

Google Sheets served as a stopgap, but it quickly became clear that it was too rudimentary to capture the full complexity of a software development cycle. The team eventually returned to Jira as the primary planning tool.

Benefits as experienced in your team

Having a dedicated weekly planning session ensured that the whole team was aligned on priorities and that task assignments were transparent and agreed upon collectively. Returning to Jira, despite the initial friction, provided a clearer structure for tracking progress and managing the backlog.

Drawbacks as experienced in your team

The early instability in tooling caused some disruption to the planning process. Estimation accuracy was inconsistent throughout the project, as noted in the context of *pair programming*, some tasks were not broken down granularly enough, which made it harder to produce reliable estimates and complicated progress tracking when tasks ran over. To deal with this problem, we added buffer time before each submission, to prevent being behind the schedule.

Lessons learned

Investing time in setting up the right tools properly at the start of a project pays off significantly in the long run. The detour through Google Sheets was a useful reminder that informal solutions rarely scale well. On estimation, the experience reinforced the importance of breaking tasks down into smaller, well-defined units, not only for

accuracy, but because smaller tasks are easier to manage, easier to hand over, and less likely to become blockers if something goes wrong.

3. Customer Tests

“As part of selecting each desired feature, the customers define automated acceptance tests to show that this feature is working”

NOTE – This principle cannot be applied in your team, as you do not have a customer on your team. If you have an opinion on whether this would have been beneficial, please add it here.

As noted, this principle was not applicable to our project as we did not have a customer on the team. However, based on prior experience, the team has some reflections on what having a customer might have meant in practice.

On one hand, having a customer define requirements directly would have removed the need to speculate about the problem domain, reducing the risk of building features that fall outside what is actually needed. On the other hand, working with non-technical stakeholders often introduces its own challenges: requests that are difficult to translate into technically sound solutions, or constraints that can gradually undermine good design and programming practices.

Managing that communication would have been a valuable but demanding exercise. Much like crafting effective prompts when working with generative AI tools, engaging with a customer requires asking the right questions, framing problems carefully, and avoiding the temptation to take requirements at face value without interrogating them. It is a skill in itself.

Finally, it is worth noting that customer satisfaction is not always a reliable indicator of project quality. User feedback, particularly around accessibility, is invaluable and should always be taken seriously. At the same time, feasibility assessments are essential, and part of the developer's role is to patiently explain when what a customer expects may not be achievable in the way they envision it.

4. Simple Design

“The team keeps the design exactly suited for the current functionality of the system. It passes all the tests, contains no duplication, expresses everything the authors want expressed, and contains as little code as possible”.

How was this applied in your team?

Simple design was not always applied instinctively, there were moments where the team got caught up in the enthusiasm of adding as many features as possible, almost as a personal challenge, which at times led to losing focus and overcomplicating things beyond what was actually required. However, the team developed a habit of flagging these situations when they arose: when someone noticed the complexity was getting out of hand, time was set aside for a group discussion to collectively assess

whether the problem at hand was actually within the scope of the system, and whether the effort being invested was justified.

One concrete example was the handling of ground staff members, which was initially becoming overly complex. After discussing it as a team, we decided to implement simple utility APIs to provide the data needed for testing and other purposes, deliberately sidestepping the bureaucratic processes that would exist in a real-world scenario but were unnecessary for our system's scope.

Benefits as experienced in your team

Catching complexity early through open team discussion prevented rabbit holes from becoming too costly. The decision to simplify, such as in the ground staff example, saved significant time and kept the codebase more focused and maintainable.

Drawbacks as experienced in your team

The tendency to over-engineer was a recurring temptation, particularly when the team was motivated and engaged. Not every instance of scope creep was caught early, and in some cases the enthusiasm for adding features came at the cost of focus on core functionality.

Lessons learned

The instinct to build more than what is needed is natural, especially in a motivated team, but it needs to be actively managed. In future projects, we would establish clearer criteria for what falls within scope at the start of each iteration, making it easier to recognise and push back on unnecessary complexity before it becomes embedded in the codebase.

5. Pair Programming

“All production software is built by two programmers, sitting side by side, at the same machine”.

How was this applied in your team?

Pairs were rotated weekly, with the previous secondary developer becoming the primary on the task and a new partner being assigned. This meant that over the course of the project, most team members had the opportunity to work across different areas of the codebase.

Benefits as experienced in your team

The rotation system ensured that knowledge was shared across the team, even if not every member touched every part of the project, most had the opportunity to work across different areas and learn from different partners. Having a second person on each task also helped catch issues earlier and kept decision-making grounded.

Drawbacks as experienced in your team

The weekly rotation created some awkward situations in practice. When tasks were short, the secondary developer didn't always get the chance to take the primary role

before the work was completed. On the other hand, when a pair had been deeply immersed in a complex problem and were close to a solution, it felt counterproductive to rotate, so on occasion the same pair extended their collaboration by an extra day rather than hand over mid-problem. This was a pragmatic call but meant the rotation wasn't always strictly followed.

Lessons learned

Rigid time-based rotation doesn't always align well with the natural rhythm of software tasks. In future projects, we would consider a more flexible approach, rotating at task boundaries rather than weekly, to ensure the secondary developer always gets a meaningful turn in the primary role without disrupting momentum at critical moments. It is also worth reflecting on whether some of these issues stemmed from uneven task sizing: better upfront estimation and splitting of tasks could help ensure pairs have enough time to rotate roles meaningfully within a single task.

6. Test-Driven Development

“The programmers work in very short cycles, adding a failing test, then making it work”

How was this applied in your team?

TDD was not applied consistently from the start, initially it was easy to forget to write tests before implementing new functionality, largely due to ingrained habits from years of writing code without a test-first approach. Over time, the team made a conscious effort to enforce TDD for each new feature added to the codebase.

Benefits as experienced in your team

As the codebase grew and new features were added or existing ones modified, the value of having a solid test suite became increasingly apparent. Tests that had felt like overhead early on proved genuinely useful later, catching regressions and giving the team more confidence when making changes to parts of the code that other features depended on.

Drawbacks as experienced in your team

The initial adoption was difficult. Without a strong habit of writing tests first, it required active effort and discipline to follow the TDD cycle consistently, and there were moments, especially early in the project, where it was skipped under time pressure.

Lessons learned

TDD is a habit that requires deliberate practice to build, and starting a project without enforcing it from day one makes it significantly harder to adopt later. In future projects, we would treat test writing as a non-negotiable part of the definition of done for each task from the very beginning, rather than something to retrofit gradually.

7. Design Improvement

“Don’t let the sun set on bad code. Keep the code as clean and expressive as possible”

How was this applied in your team?

Refactoring was typically carried out after a feature had been implemented and its core functionality verified. The team adopted a deliberately two-phase approach: an initial, rougher implementation focused on getting the logic right and understanding the direction, followed by a cleanup pass once the overall picture was clearer. This was a conscious choice, code was not written carelessly from the start, but the team accepted a degree of roughness in the first pass knowing that refinement would follow once the shape of the solution was better understood.

Benefits as experienced in your team

This approach allowed the team to move quickly during the exploratory phase without getting bogged down in premature optimisation, while still ensuring the final code was clean and well-structured. Refactoring with a clearer understanding of the problem also tended to produce better design decisions than trying to get everything right on the first attempt.

Drawbacks as experienced in your team

Touching existing code occasionally introduced bugs. However, thanks to the test suite built up through TDD, these were caught promptly and resolved without significant disruption, reinforcing the value of having reliable tests in place before refactoring.

Lessons learned

The combination of a deliberate two-phase implementation approach and a solid test suite made refactoring a manageable and productive part of the workflow rather than a source of risk. The experience confirmed that refactoring and TDD are most effective when practiced together, tests provide the safety net that makes improving code feel safe rather than dangerous.

8. Continuous Integration

“The team keeps the system fully integrated at all times”

How was this applied in your team?

The team practised continuous integration in a direct fashion: there was no separate testing or staging environment, so code moved straight from local development to production. Whenever a major feature (or a group of small features) was completed, a new release was pushed and the production environment was updated immediately. This meant that the live version of the system always reflected the team's most recent work.

Benefits as experienced in your team

Having every completed feature visible in production almost immediately was one of the most rewarding aspects of the project. It gave the team a tangible sense of progress, rather than working in isolation and hoping everything would come together at the end, each pair could see the results of their work reflected in the live web and mobile application. This also made it significantly easier to follow what other pairs had been doing: instead of reading through pull requests or waiting for a handover, team members could simply open the application and see the changes for themselves. From a practical standpoint, the approach also simplified the weekly pair rotations. When a new partner picked up a task, they could pull the latest working code and have a functioning system to build on, which reduced the overhead of context-switching between pairs.

Drawbacks as experienced in your team

No drawbacks.

Lessons learned

The experience reinforced that continuous integration is not just a technical practice but also a motivational one: seeing work go live regularly kept the team engaged and gave each session a clear sense of purpose. It also highlighted the importance of every team member understanding the code and the structure of the software thoroughly.

9. Collective Code Ownership

“Any pair of programmers can improve any code at any time”

How was this applied in your team?

The team worked on separate branches and consulted each other before merging code into main, ensuring the main branch remained stable and clean at all times. Collective ownership was supported in practice by the pair programming rotation, which gave most team members exposure to different parts of the codebase over the course of the project.

Benefits as experienced in your team

The branching strategy and pre-merge communication meant that code changes were rarely a surprise to the rest of the team, and the main branch could generally be trusted as a stable reference point. The pair rotation further helped distribute knowledge across the codebase, making it easier for multiple people to contribute to or review different areas.

Drawbacks as experienced in your team

As noted in earlier sections, not every part of the codebase was touched by enough pairs, meaning that for certain components, only the developers who had originally

worked on them had sufficient familiarity to address issues when they arose. In practice, collective ownership was therefore more complete in some areas than others.

Lessons learned

Collective ownership is difficult to achieve fully without deliberate effort to ensure knowledge is spread evenly. In future projects, we would pay closer attention to identifying "knowledge silos" early and actively rotating pairs into those areas before they become bottlenecks.

10. Coding Standard

“All the code in the systems looks as if it was written by a single – very competent – individual”

How was this applied in your team?

Coding standards were maintained primarily through the consistent use of generative AI tools, specifically Claude, to assist with code writing. This was particularly valuable given that not all team members were equally familiar with every language and framework used in the project, including Go and Flutter among others, making AI assistance a natural way to bridge knowledge gaps while keeping the code broadly consistent in style and structure. Additionally, working in pairs helped the team develop a shared approach to how prompts were crafted, which indirectly contributed to more consistent output over time.

Benefits as experienced in your team

The use of a single AI tool as a common coding assistant acted as an informal style enforcer, producing code that was broadly consistent in structure and naming conventions without requiring the team to explicitly define and police a standard. For team members less familiar with a particular language or framework, it also served as a learning resource, helping them write code that conformed to conventions they might not yet have been fully aware of.

Drawbacks as experienced in your team

The team did not implement any formal code review process with respect to style, nor did we make explicit use of automated formatting tools such as *gofmt*, which is available natively in Go. Consistency was essentially assumed rather than verified, which left some room for stylistic drift going unnoticed.

Lessons learned

Relying on AI-generated consistency is a reasonable starting point but is not a substitute for explicit standards and automated enforcement. In future projects, we could integrate a formatter like *gofmt* from the start and include a basic style check as part of the code review process, rather than trusting that consistency would emerge naturally.

11. Metaphor

“The team develops a common vision of how the program works”

How was this applied in your team?

The first semester was spent almost entirely on planning and framing the scope of the software: defining use cases, establishing the technical architecture, and mapping out the functional architecture. At the time, this phase felt tedious, particularly when the team was eager to start building. However, it proved to be one of the most valuable investments of the project. By the time development began, every team member had a shared understanding of how the system was structured, what each component was responsible for, and how the pieces fit together. This common vocabulary and mental model carried through the entire development phase. Pair programming reinforced this further: because team members rotated across different parts of the codebase, the shared vision was continuously refreshed and corrected through conversation, rather than drifting as it might have if individuals had worked in isolation.

Benefits as experienced in your team

Having a common mental model of the program made communication within the team significantly more efficient. When discussing a problem or proposing a change, there was rarely a need to explain the broader context from scratch, everyone already understood where a given component sat within the system and what its responsibilities were. This shared understanding also made it easier to identify points of friction during discussions: when two people disagreed, the common vocabulary allowed the team to pinpoint the source of the disagreement quickly rather than talking past each other. In practice, this meant that the team's attention and effort were directed at solving genuine technical problems rather than resolving misunderstandings about how the system was supposed to work.

Drawbacks as experienced in your team

The main cost was the slow start. Spending an entire semester on planning before writing any production code required patience, and there were moments where it felt like progress had stalled. However, the foundation this phase created was strong enough that the team did not experience any significant drawbacks from it during development. In hindsight, the trade-off was clearly worthwhile.

Lessons learned

The experience reinforced that time spent building a shared understanding of a system before development begins is not wasted, even when it feels that way in the moment. A lot of documentation writing in the beginning was necessary as we referred to it regularly during the second semester.

12. Sustainable Pace

“The team is in it for the long term. They work hard, at a pace that can be sustained indefinitely. They conserve their energy, treating the project as a marathon rather than a sprint”

How was this applied in your team?

The team developed a natural rhythm for its working sessions that helped maintain a sustainable pace throughout the semester. A typical session began with roughly half an hour of informal discussion: each pair would outline their task for the day, clarify goals, and ease into the work progressively. This was followed by a focused block of around two hours of concentrated development. In the final half hour, the task was either completed or had surfaced a conflict or unexpected problem within the software. If the fix was straightforward, it was addressed in the remaining time; if not, it was framed as a separate task to be picked up in a future session rather than pushed through at the cost of quality and taking the risk or creating more problems. Before submission deadlines, the workload inevitably increased, but it remained manageable.

Benefits as experienced in your team

Framing each session around a clearly defined task for the day helped the team maintain focus without overcommitting. The discipline of deferring unresolved problems to another day, rather than forcing a solution under fatigue, preserved both code quality and team energy over the course of the project. That said, tasks frequently took longer than initially expected... It was a recurring pattern but as we had 5 months to build the software, taking one more day for a task was feasible. In the long term we knew we would take more time so we added buffer zones into the planning to mitigate this issue.

Drawbacks as experienced in your team

None.

Lessons learned

The contrast between semesters was instructive. In the first semester, the team had a six-hour session on Fridays, which consistently felt too long: concentration and productivity dropped noticeably in the later hours. This semester, sessions were capped at roughly three and a half hours, and the difference was striking: the shorter window forced better time management and kept energy levels higher throughout. The experience reinforced that productivity is not proportional to hours spent, and that shorter, well-structured sessions are far more effective than longer ones where focus gradually erodes.

13. Overall Project

Benefits as experienced in your team

The project delivered benefits that went well beyond the technical. On a personal level, it was genuinely enjoyable and built lasting relationships within the team. The experience also taught the team to recognise and accept different working styles: some members needed to talk through their ideas before writing any code, while others could only articulate their thinking once they had started building something.

From a professional development perspective, the project provided an unusually effective environment for learning. The combination of pair programming, weekly rotations, and constant collaboration meant that knowledge transfer happened continuously and organically. Skills and techniques that might have taken weeks to pick up through individual study were absorbed in days simply by working alongside someone who already understood them. This aspect of the project felt like a genuine preview of what effective teamwork looks like in a professional setting.

Drawbacks as experienced in your team

The main drawback was the social energy required.

Lessons learned

The most valuable lesson was learning how to work in a group without taking feedback or criticism personally. In a fast-paced collaborative environment, remarks are not always delivered with perfect diplomacy, tasks need to progress, decisions need to be made, and sometimes the phrasing of a disagreement reflects the pressure of the moment rather than any ill intent.

14. Analysis of the use of Generative AI tools in your project, as experienced in your team

Generative AI tools, primarily Claude, Claude code, Gemini and ChatGPT, were used throughout the project in a variety of ways.

Strengths of Gen AI tools as experienced in your team across the whole development lifecycle (where relevant). Please give explicit examples.

The most frequent use was in code generation. When working on repetitive or boilerplate-heavy tasks, prompting an AI tool to produce an initial draft saved considerable time.

AI tools were also used to generate and compare methodologies. When the team faced design decisions, such as how to structure the scheduling logic or which approach to take for handling concurrency, prompting a model to outline two or three possible strategies, along with trade-offs, served as a useful springboard for team discussion. Rather than replacing deliberation, it helped frame it more efficiently.

A third area of value was in seeking advice on architecture. Early in the project, the team used AI tools to explore how similar systems are typically structured, asking questions about layering, separation of concerns, and service boundaries. This was especially helpful during the initial design phase, where the team lacked strong prior experience in building a system of this scale and needed to orient quickly.

Limitations of Gen AI tools as experienced in your team across the whole development lifecycle (where relevant). Please give explicit examples.

The quality of the output depended heavily on the quality of the prompt, which in turn depended on the existing knowledge of the person writing it. A team member who already understood the problem domain well could extract genuinely useful suggestions, while a vague or poorly scoped prompt often produced generic or misleading code that took longer to debug than it would have taken to write from scratch.

A practical constraint was that credits were exhausted quickly, particularly on tools with usage-capped free tiers or limited API access. During intensive development periods, it was not uncommon for a team member to hit their daily or weekly limit mid-task, forcing a switch back to manual problem-solving or reliance on documentation.

Finally, the tools had no access to our cloud implementation or the specifics of our deployed infrastructure. Any question involving our actual configuration, environment variables, deployment pipeline, or runtime behaviour required the team to either provide extensive context manually or, more often, fall back on direct debugging. This limited the usefulness of AI tools for the later stages of the project, where most issues were environment-specific rather than conceptual.

Lessons learned

The most important takeaway was that generative AI is a productive tool only when the developer understands what is being generated. When used by someone with sufficient context, it accelerates routine work and supports decision-making. When used without that understanding, it becomes counterproductive: the developer risks integrating code they cannot maintain, debug, or explain to a pair partner. In future projects, we would continue using AI tools selectively but would emphasise that each team member should be able to fully justify and explain any AI-assisted output before it enters the codebase.